

MLPC 2026 Task 5: Challenge Report

Waji Ul Hassan Syed Rayan Ali Javed
Team Stochastic Pipelines

June 24, 2026

1 Baseline Reproduction

(a) We reproduced the provided baseline: one `DecisionTreeClassifier` (`max_depth=20`, `max_features='sqrt'`) per class wrapped in a `MultiOutputClassifier`, trained on 50,000 subsampled segments using the raw 960 features (no scaling, no PCA). For inference on the non-hidden test set (500 files), we predicted on overlapping 1-second segments (hop 0.5 s), kept only whole-second predictions, and merged consecutive active seconds into onset/offset intervals. The official `evaluate.py` script yields a Segment-based Macro F1 of **0.3170**. Per-class results are reported in Table 1.

(b) Performance varies widely, from $F1 = 0.061$ (`window_open_close`) to 0.578 (`running_water`). Long, continuous sounds score highest because they occupy many stable 1-second segments, whereas short transients suffer from the coarse 1-second resolution and the discarded half-second predictions. Class frequency helps but is not decisive: footsteps ($\approx 10\%$ of segments) reaches only 0.333, while the less frequent phone_ringing reaches 0.485, indicating that acoustic ambiguity and label noise often dominate. Mechanical impacts (door, wardrobe, window) share broadband transient spectra and are mutually confused ($F1$ 0.061–0.177). Footsteps additionally overlaps with background transients and suffers from low annotator agreement (84.82% in Task 3). Precision generally exceeds recall, showing a conservative model that avoids false alarms but misses true events, consistent with shallow per-class trees.

(c) The baseline is simple, interpretable, and fast to train and run; it handles polyphonic audio via binary relevance and reuses precomputed acoustic features. Its main limitations are: independent per-class classifiers cannot model class co-occurrence; the 1-second resolution and discarded half-second predictions miss short transients; single unregularised trees capture only axis-aligned boundaries and struggle to separate acoustically similar classes; feature aggregation over one second discards fine-scale dynamics; and there is no temporal modelling, so predictions are noisy across time. Majority voting also propagates label noise for low-agreement classes. The baseline works well for long, distinctive sounds (`running_water`, `vacuum_cleaner`) and poorly on short, ambiguous, or noisy-label events (`window_open_close`, `wardrobe_drawer`), motivating a stronger classifier.

Table 1: Per-class baseline precision, recall, and F1 on the non-hidden test set, and the F1 of our Random Forest with the absolute change Δ . Macro F1: baseline 0.3170, Random Forest 0.4714.

Class	Base P	Base R	Base F1	RF F1	Δ
bell_ringing	0.426	0.218	0.288	0.207	-0.081
coffee_machine	0.504	0.283	0.362	0.449	+0.087
cutlery_dishes	0.294	0.281	0.287	0.489	+0.202
door_open_close	0.200	0.159	0.177	0.370	+0.193
footsteps	0.384	0.293	0.333	0.488	+0.155
keyboard_typing	0.372	0.347	0.359	0.570	+0.211
keychain	0.365	0.287	0.321	0.496	+0.175
light_switch	0.240	0.228	0.234	0.484	+0.250
microwave	0.408	0.373	0.390	0.609	+0.219
phone_ringing	0.534	0.444	0.485	0.559	+0.074
running_water	0.655	0.517	0.578	0.695	+0.118
toilet_flushing	0.288	0.292	0.290	0.521	+0.231
vacuum_cleaner	0.485	0.455	0.470	0.637	+0.167
wardrobe_drawer_open_close	0.134	0.109	0.120	0.292	+0.172
window_open_close	0.057	0.066	0.061	0.206	+0.145

2 Simple Classifiers

(a) Our best classical classifier is a Random Forest, implemented via `MultiOutputClassifier` (binary relevance, one model per class) for polyphonic SED. We chose it over single decision trees because: (i) bagging many trees on bootstrapped samples reduces variance and overfitting; (ii) random feature selection at each split decorrelates the trees and improves generalisation; (iii) `class_weight='balanced'` upweights minority classes, which is critical given that the rarest classes occupy under 2% of segments; and (iv) `max_depth` and `min_samples_leaf` provide explicit regularisation. The final model uses `n_estimators=200`, `max_depth=20`, `min_samples_leaf=5`, and `class_weight='balanced'`, trained on the same 50,000 segments with raw features (no scaling, no PCA, matching the baseline for a fair comparison). Trees are scale-invariant, so scaling and PCA are unnecessary and can even harm axis-aligned splits.

(b) We replaced the decision trees with the Random Forest and studied the effect of `max_depth` and `min_samples_leaf` on validation Macro F1, with `n_estimators=200` and balanced weighting fixed (Figure 1). Tuning was performed on the local validation set only, never the non-hidden test set. Raising `min_samples_leaf` from 1 to 5 helped at every depth (e.g. 0.4633 to 0.4818 at depth 20). Performance rose from depth 10 to 20 (0.4720 to 0.4818) as deeper trees captured more complex patterns, then fell at depth 30 (0.4512) from overfitting. We selected `max_depth=20`, `min_samples_leaf=5` (validation Macro F1 0.4818).

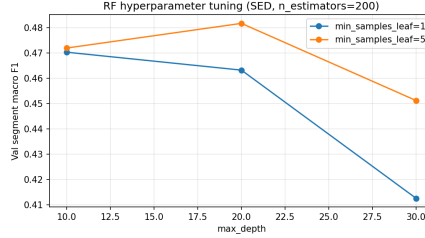


Figure 1: Random Forest tuning: validation Macro F1 versus `max_depth` for `min_samples_leaf` $\in \{1, 5\}$ (`n_estimators`=200, balanced).

(c) On the non-hidden test set, the Random Forest raises Macro F1 from 0.3170 to **0.4714** (+0.1544), evaluated with the same official metric and pipeline as the baseline (Table 1). The largest gains are on previously weak classes: `light_switch` (+0.250), `toilet_flushing` (+0.231), `microwave` (+0.219), `keyboard_typing` (+0.211), and `cutlery_dishes` (+0.202). Balanced weighting plus ensemble stability recovers rare and mid-frequency classes that a single tree failed on. The hard mechanical-transient classes improve in relative terms but remain lowest: `window_open_close` (0.206) and `wardrobe_drawer_open_close` (0.292), reflecting the 1-second resolution limit. One class regresses: `bell_ringing` drops -0.081 (0.288 to 0.207), likely because balanced weighting makes the model over-predict competing rare classes in those segments, compounded by its scarcity and tonal similarity to `phone_ringing`.

(d) We analysed two test files (Figure 2). In `000790.wav`, `running_water` is detected almost perfectly (14 of 15 true seconds, no false alarms), confirming that continuous, energetic sounds are handled well. However, `microwave` (TP = 4, FN = 10) and `phone_ringing` (TP = 3, FN = 6) show low recall: the model fires on their salient frames but misses the quieter sustained or interring portions. Three false `cutlery_dishes` seconds appear, a mild instance of balanced-weighting over-prediction. In `005294.wav`, `running_water` (no false alarms) and `toilet_flushing` are again caught, but the mechanical transients are missed or smeared: `wardrobe_drawer` (5 of 8 true seconds missed) and `door_open_close` (FP = 3), with extra false positives on footsteps (FP = 4) and other classes. These two files capture all our error modes: reliable detection of continuous sounds, low recall on intermittent sounds, confusion among short mechanical transients, and over-prediction from balanced weighting.

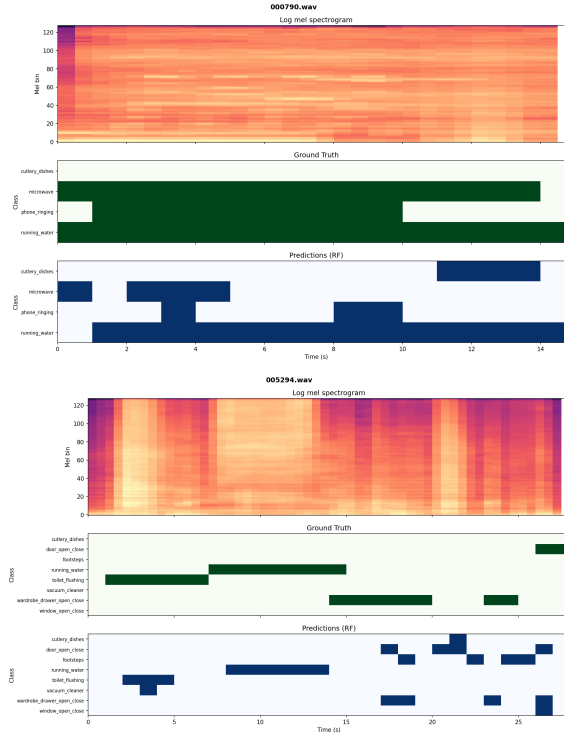


Figure 2: Qualitative error analysis: log-mel spectrogram (top), ground truth (middle), and Random Forest predictions (bottom) for each file. *Top block* 000790.wav: running_water is detected reliably, while microwave and phone_ringing show low recall. *Bottom block* 005294.wav: continuous sounds are caught, while short mechanical transients are missed or confused and several false positives appear.

3 Post-Processing and Temporal Refinement

(a) Each 1-second segment is classified on its own, so a class’s predicted activity can flicker over time. Isolated active seconds appear, and short gaps open inside otherwise continuous events. The task allowed either temporal smoothing of confidence scores or median filtering of the binary predictions. We used median filtering. Our classifier outputs hard 0/1 decisions rather than calibrated scores, and a median is the natural operator for deleting short spurious activations while keeping longer events intact. We ran the filter on each class’s per-second prediction sequence, replacing every value with the median of a window centred on it. We applied it within each recording and each class, on the whole-second prediction matrix and before merging predictions into intervals. Smoothing therefore stays inside a single file’s timeline and never crosses file boundaries. We varied the one key parameter, the window size, across $\{1, 3, 5, 7, 9\}$ seconds,

treating window 1 as the no-filtering reference, and scored every setting on the non-hidden test set with the official Segment-based Macro F1.

(b) Filtering made things worse. Macro F1 fell steadily from 0.4714 without filtering to 0.3638 at window 9 (Table 2, Figure 3), so the best setting is no post-processing, and we generate the final hidden-test predictions without it. The reason is the shape of our errors, not a flaw in the method. A median filter only helps when mistakes are short, isolated false activations sitting inside otherwise correct predictions. Our main mistake is the opposite: low recall on intermittent events, with microwave caught in 4 of 14 seconds and phone_ringing in 3 of 9. When real detections are already sparse and broken (for example 0 1 0 0 1 0), the median deletes the true detections instead of cleaning up noise, and recall drops further. Event length makes this worse. Many targets such as door, wardrobe, and window last only 1–2 seconds, and any window of 3 or more wipes out events shorter than half its length. Macro F1 weights every class the same, so erasing these short rare-class events drags the average down sharply, which is why the loss grows with the window. The decline is smooth and consistent across all five settings, so this is a structural mismatch rather than measurement noise. We keep the unfiltered Random Forest as our final system. The result is still informative: temporal smoothing pays off only when the leftover errors match what the operator assumes, and ours do not. A learned temporal model that decides when to bridge a gap would fit our error pattern far better than a fixed window that treats every gap alike.

Table 2: Effect of median filter window size on non-hidden test Macro F1.

Window (s)	1 (none)	3	5	7	9
Macro F1	0.4714	0.4432	0.4071	0.3842	0.3638

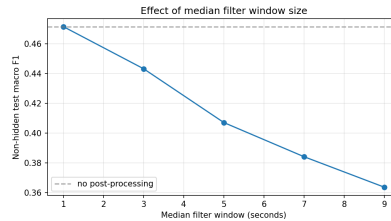


Figure 3: Non-hidden test Macro F1 versus median filter window size. The dashed line marks the no-filtering reference.

4 Reflection and Real-World Considerations

(a) Our main limitations follow directly from the errors we observed. First, short mechanical transients are confused with one another. The three weakest classes, window (0.21), wardrobe (0.29), and door (0.37), share broadband

transient spectra, and because each segment is judged alone the model has no surrounding context to tell them apart. A model that reads several neighbouring segments together, such as a CRNN, could use that context to separate them. Second, intermittent events are under-detected. Microwave and phone_ringing are found only on their loudest seconds because per-second classification knows nothing about event continuity. A sequence model that carries state over time, or a learned smoothing stage like a CRF instead of a fixed median, would close these gaps without deleting genuine detections. Third, the temporal grid is coarse. The 1-second resolution and the dropped half-second segments hurt short events, and bringing the overlapping 0.5 s predictions back in, or extracting features at a finer hop, is a concrete way to recover them. Fourth, rare classes and noisy labels cap performance. light_switch lacks training data and footsteps carries label noise (84.82% agreement), so more rare-class data and noise-tolerant training would help. Fifth, balanced weighting over-predicts. It lifted rare-class recall but added false positives, and a per-class threshold tuned on validation, rather than a flat 0.5, could win back precision without losing those gains. Finally, the model ignores class co-occurrence. Binary relevance scores the 15 classes independently and cannot use patterns like cutlery_dishes appearing alongside running_water in a kitchen; a shared multi-label model could.

(b) In its current form the system is not reliable enough to ship as a smart-home assistant, although some parts are encouraging. It is cheap to run: a Random Forest over precomputed 1-second features works on modest hardware with no GPU, and the 1-second design fits near-real-time, low-latency use on an embedded device. The problem is accuracy. A Macro F1 of 0.4714 means many events are missed or wrongly fired, and in a home a missed “running water” or a false “door opened” alert loses the user’s trust fast. Robustness is also untested. Real rooms add reverberation, overlapping speech, music, and microphone differences that our training data does not fully cover, and we already saw running_water missed in one file whose sound differed from training. The over-prediction tendency would show up as frequent false alarms, which is the quickest way to make someone switch a feature off. The system also depends on its exact feature front-end, so any change in how audio is captured would degrade it. In short, it is an efficient proof of concept, but it would need stronger temporal models, per-class threshold calibration, and testing on realistic in-home audio before anyone could rely on it.

Disclosure of LLM and AI Tool Use

We used Claude (Anthropic) for code generation (the SED inference pipeline, evaluation wrappers, hyperparameter sweeps, and matplotlib visualisations), for brainstorming the post-processing analysis, and for improving the clarity of this report. We also used it to cross-check both team members’ notebooks for methodological consistency (ensuring all results use the official `evaluate.py` metric and that hyperparameters were tuned only on the validation set). All

code was verified cell by cell against expected outputs, and every reported number was reproduced on our shared pipeline before inclusion. No content was included without being understood and verified by the authors.